



Universidad de Cundinamarca
Facultad de Ingeniería
Minería de Datos

Informe Técnico — Etapa 3
Tratamiento de datos proceso ETL en SQL Server Integration Services (SSIS)

<https://github.com/DROMEROH14/migracion-desplazamiento-humano>

Estudiante: David Santiago Romero
Docente: Édison Gustavo Cañón Varela
Proyecto: Migración y Desplazamiento Humano

Septiembre 29 de 2026

1. Introducción y objetivo

Este informe documenta la implementación y verificación del proceso ETL (Extracción, Transformación y Carga) construido con SQL Server Integration Services (SSIS) para tratar los problemas de calidad de datos identificados en la Etapa 2 del proyecto Migración y Desplazamiento Humano. El proceso parte del dataset consolidado de 28.467 registros (Etapa 1) y del diagnóstico de calidad en seis dimensiones —completitud, exactitud, consistencia, unicidad, validez y actualidad— obtenido en la Etapa 2.

El objetivo concreto es doble: primero, aplicar reglas de tratamiento trazables sobre cada problema ya diagnosticado, usando los componentes estándar de un Data Flow de SSIS (Derived Column, Data Conversion, Lookup, Conditional Split y Sort); segundo, demostrar —con tres iteraciones reales y una prueba explícita— que el proceso no pierde información, que separa correctamente los registros que requieren revisión manual conservando su valor original, y que es idempotente: volver a ejecutar el mismo lote no genera duplicados en el destino.

Todo el proceso se apoya en una zona de staging (etl.stg_desplazamiento) que conserva el dato exactamente como llega, sin tipar ni transformar, lo que garantiza trazabilidad y repetibilidad completas: si en el futuro se necesita auditar por qué un registro terminó en la tabla de revisión, siempre se puede volver al dato crudo del staging.

2. Diagnóstico inicial y reglas de tratamiento

La Etapa 2 midió el dataset consolidado en seis dimensiones de calidad y produjo un inventario de problemas con evidencia cuantitativa (por ejemplo: 345 registros sin valor en personas_desplazadas, exactitud del 98.79% por valores negativos, y un catálogo de dominio de sexo que resultó incompleto una vez confrontado con los datos reales). Cada regla de tratamiento definida a continuación responde directamente a uno de esos hallazgos: no se trata de validaciones genéricas, sino de reglas trazables hasta el problema que las originó.

Campo	Problema de origen (Etapa 2)	Acción en el Data Flow	Justificación
personas_desplazadas	345 registros sin valor reportado (Completitud, Etapa 2)	Conditional Split: si es nulo, se enruta a la tabla de revisión con motivo SIN_DATO_ORIGEN. No se imputa con 0.	Imputar con 0 subestimaría el fenómeno; la fuente simplemente no reportó cifra para ese país-año-causa. Se documenta, no se corrige.
personas_desplazadas / desplazamiento_stock	Exactitud (Etapa 2, 98.79%)	Derived Column valida que el valor sea ≥ 0 ; si es negativo, se enruta a revisión con motivo VALOR_NEGATIVO.	Un valor negativo de personas desplazadas no tiene sentido físico; es preferible aislarlo y revisarlo manualmente que dejarlo pasar.
año, personas_desplazadas, desplazamiento_stock	Tipos de dato mixtos entre fuentes (texto/número)	Data Conversion fuerza año a entero y los montos a decimal; si la conversión falla, Conditional Split enruta a revisión con motivo ERROR_CONVERSION_TIPO.	Sin una conversión explícita y controlada, un valor no

Campo	Problema de origen (Etapa 2)	Acción en el Data Flow	Justificación
			numérico rompe los cálculos agregados aguas abajo sin avisar.
sexo	Catálogo de valores válidos incompleto (detectado en la Iteración 1)	Lookup contra la tabla de referencia ref_sexo; en la Iteración 1 el catálogo solo tenía 3 de los 4 valores reales y generó 100 rechazos evitables; se corrigió en la Iteración 2.	El Lookup evita valores inventados o mal escritos, pero el catálogo de referencia debe reflejar el dominio real de la fuente, no una suposición inicial.
nivel / departamento / municipio	Consistencia jerárquica (Etapa 2, 100%, pero se conserva como control activo)	Derived Column valida que un registro Regional tenga departamento/municipio y que uno Global/Nacional no los tenga; si falla, se enruta a revisión con motivo INCONSISTENCIA_NIVEL_GEOGRAFIA.	Aunque hoy no hay inconsistencias, dejar el control activo evita que una carga futura las introduzca sin que nadie se entere.
año	Validez de rango temporal	Derived Column valida que el año esté entre 1970 y el año en curso; si no, se enruta a revisión con motivo ANIO_FUERA_DE_RANGO.	Un año fuera de ese rango normalmente indica un error de digitación o de mapeo de columnas entre fuentes.
fila completa (todas las columnas)	Riesgo de duplicados al recargar el mismo dataset o un dataset que se superpone con cargas anteriores	Se construye una llave de negocio (nivel+iso3+departamento+municipio+año+causa+sexo+grupo_edad+fuente); un Lookup contra el destino separa, con Conditional Split, los registros que ya existen (no se reinsertan) de los realmente nuevos.	Permite ejecutar el mismo paquete varias veces (por ejemplo, si se agrega un año nuevo a la fuente) sin duplicar todo el histórico ya cargado.
personas desplazadas (valores atípicos)	3.222 registros fuera de 3xIQR dentro de su nivel (Etapa 2)	El esquema reserva la columna outlier_iqr_extremo (BIT, default 0) en la tabla destino para marcar estos casos sin descartarlos; el cálculo automático de esta bandera se validó en la simulación Python (etl_simulation.py) y queda documentado como siguiente incremento del Derived Column.	Los valores extremos corresponden a crisis humanitarias reales y documentadas (p. ej. Siria, Colombia en años de pico de conflicto);

Campo	Problema de origen (Etapa 2)	Acción en el Data Flow	Justificación
			no se descartan ni se envían a revisión, para no sesgar el análisis.

Nota metodológica: los valores nulos de personas desplazadas NO se imputan con 0. Imputar subestimaría el fenómeno cuando la fuente simplemente no reportó una cifra para ese país-año-causa; en cambio, esos registros se separan a la tabla de revisión, preservando el resto de las columnas originales para que un analista pueda decidir con contexto completo.

3. Diseño del paquete SSIS: Control Flow y Data Flow

3.1 Control Flow

El paquete Package.dtsx (Paquete_Tratamiento_Desplazamiento) organiza la ejecución en seis pasos conectados por restricciones de precedencia (precedence constraints). (1) 'Registrar Inicio de batch' — Execute SQL Task que inserta un nuevo registro en etl.log_batches y devuelve el batch_id con el que se identifica toda la corrida. (2) 'Truncar tablas temporales' — Execute SQL Task que vacía etl.stg_desplazamiento, etl.tmp_candidatos_validos y etl.tmp_revision_lote, dejando cada corrida en un estado limpio y repetible. (3) 'Data Flow Task' ('Cargar staging') — lee el archivo origen y lo escribe sin transformar en etl.stg_desplazamiento. (4) 'Transformar y cargar' — Data Flow Task que aplica todas las reglas de validación, tipado y cálculo de la llave de negocio. (5) 'Ejecutar MERGE idempotente' — Execute SQL Task que llama a etl.sp_merge_lote, el procedimiento que compara contra el destino y decide, por cada fila candidata, si es un registro nuevo (INSERT) o ya existente (UPDATE). (6) 'Marcar batch OK' / 'Marcar batch ERROR' — un branch por restricción de precedencia (Éxito / Error) que cierra el registro del lote en el log con el resultado final.

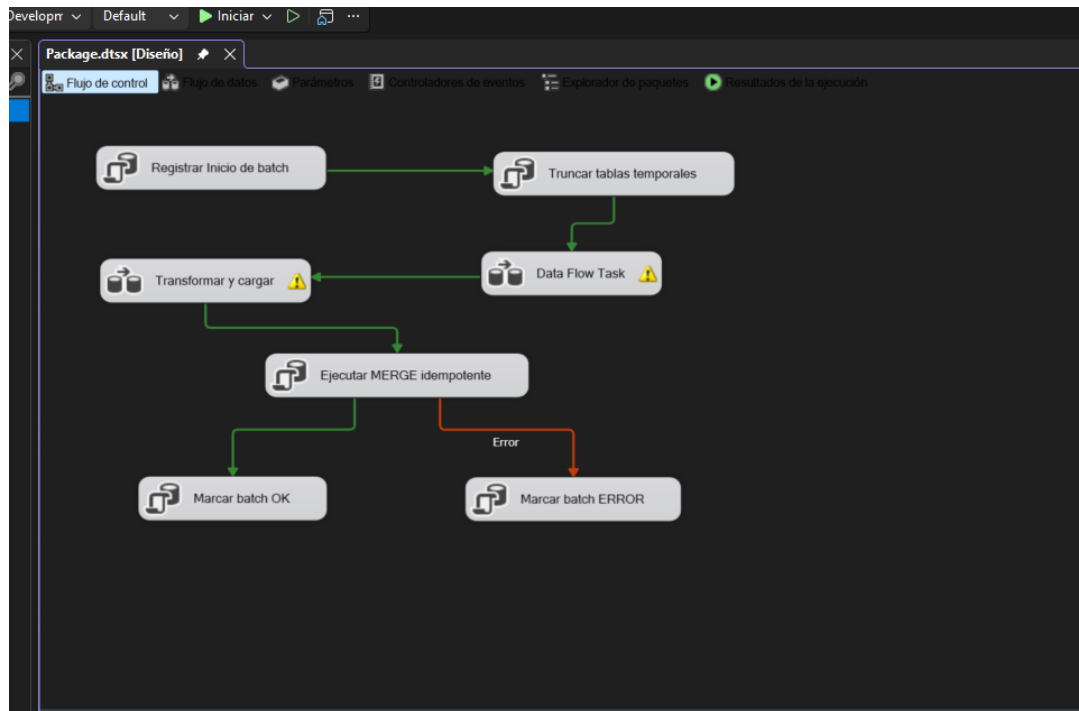


Figura 1. Control Flow del paquete, vista de diseño (captura real de SSDT/Visual Studio).

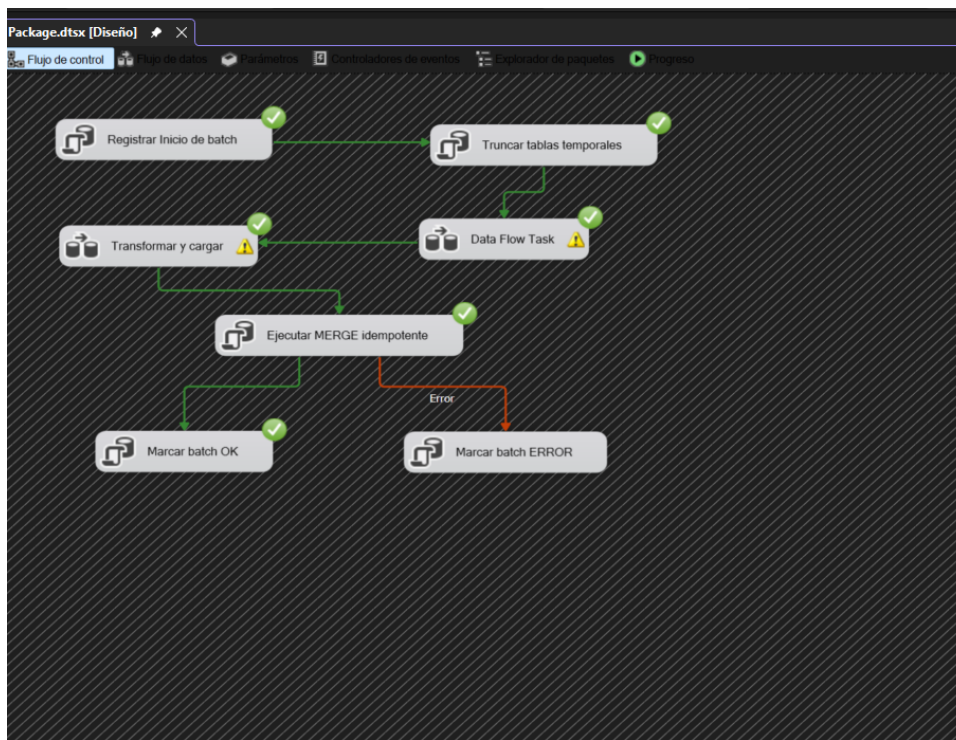


Figura 2. Ejecución exitosa del paquete completo: los seis pasos en verde, con 'Marcar batch ERROR' sin ejecutarse porque el branch de éxito fue el que se activó.

3.2 Data Flow

El Data Flow Task 'Transformar y cargar' es donde ocurre todo el tratamiento. El flujo real, ya construido y probado, es: OLE DB Source (lee etl.stg_desplazamiento) → Derived Column 'Validaciones de texto' (limpia campos vacíos y calcula banderas de validación) → Data Conversion 'Convertir tipos' (fuerza anio, personas_desplazadas y desplazamiento_stock a sus tipos definitivos) → Derived Column (calcula es_negativo, anio_fuera_de_rango y la llave de negocio clave_negocio_texto) → Lookup contra etl.ref_sexo (separa sexo válido / inválido) → Union All (reúne ambas salidas del Lookup) → Derived Column 'Calcular motivo de revisión' (resuelve un único motivo textual cuando aplica alguna regla) → Conditional Split (separa 'Validos' de 'ARevision') → Sort (ordena 'Validos' por clave_negocio) → dos OLE DB Destination, uno hacia etl.tmp_candidatos_validos y otro hacia etl.tmp_revision_lote. El Row Count conectado a la salida de error de 'Convertir tipos' es un componente de diagnóstico usado durante la construcción del paquete, sin efecto sobre el resultado final.

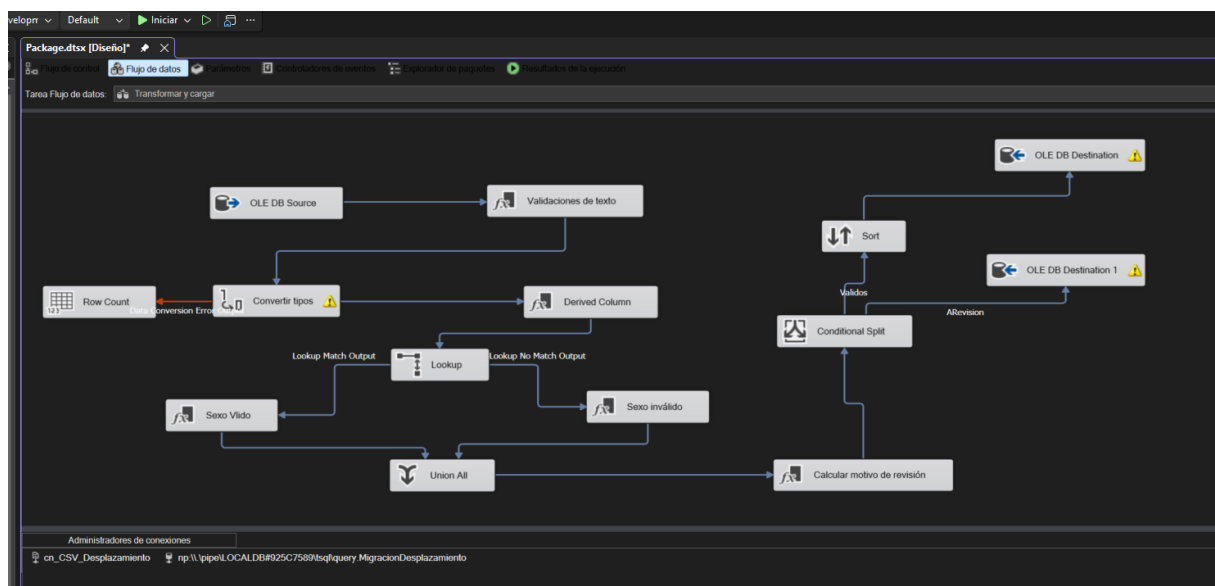


Figura 3. Data Flow 'Transformar y cargar', vista de diseño completa (captura real).

La detección de duplicados NO ocurre dentro de este Data Flow: los candidatos válidos se escriben siempre en la tabla temporal `etl.tmp_candidatos_validos`, y es el paso 5 del Control Flow (MERGE, `etl.sp_merge_lote`) el que compara `clave_negocio` contra `etl.desplazamiento_tratado` y decide INSERT (nuevo) vs. UPDATE (duplicado). Separar esa decisión del Data Flow simplificó el diseño y es lo que garantiza la idempotencia sin depender de un Lookup adicional contra el destino.

El Data Flow Task de staging es mucho más simple: solo copia el archivo origen hacia `etl.stg_desplazamiento` sin aplicar ninguna regla de negocio, preservando el dato exactamente como llega.

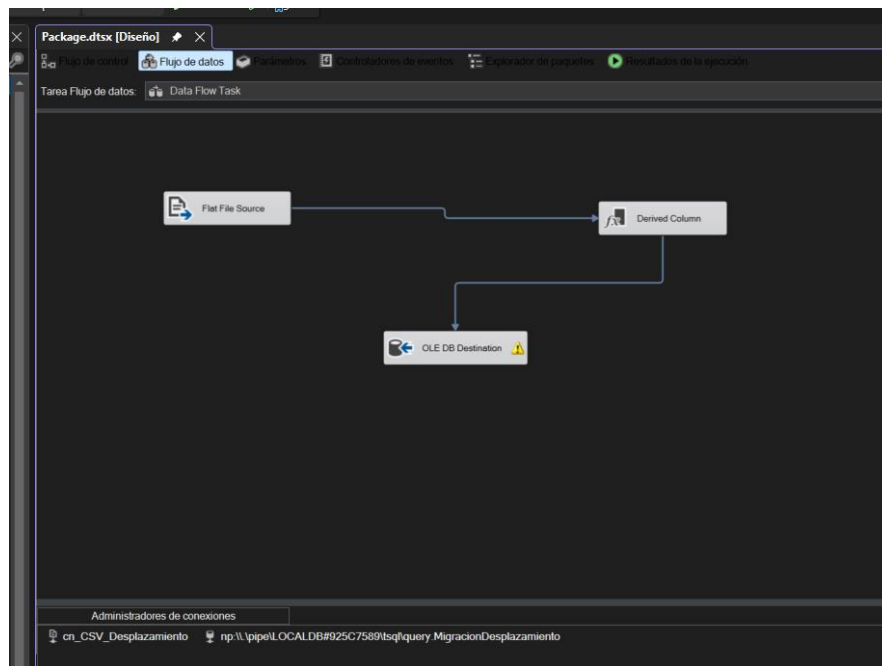


Figura 4. Data Flow de staging: Flat File Source → Derived Column (columnas de control) → OLE DB Destination (captura real).

La zona de staging cumple un rol distinto al de las tablas temporales de lote: el staging (`etl.stg_desplazamiento`) es append-only dentro de cada corrida y NUNCA se transforma — ahí queda el dato tal como llegó, columna por columna, en NVARCHAR. Las tablas `tmp_candidatos_validos` y `tmp_revision_lote`, en cambio, ya contienen el resultado tipado y validado de un lote específico, y se truncan al inicio de cada corrida. Esta separación es la que permite, ante cualquier duda, reconstruir por qué un registro terminó donde terminó.

4. Configuración de los componentes (guía de construcción)

Esta sección documenta la configuración real y ya verificada de las tareas del paquete, tal como quedó construido y probado en SSDT/Visual Studio, con captura de pantalla real de cada Execute SQL Task del Control Flow. Los componentes del Data Flow se documentan con su expresión/configuración exacta; donde aún no se anexó la captura individual del editor, se deja marcado como pendiente.

4.0 Tareas del Control Flow (Execute SQL Task)

Registrar Inicio de batch

Primer paso del paquete: inserta la fila de control del lote en `etl.log_batches` y devuelve, con OUTPUT `INSERTED.batch_id`, el identificador que usan todas las demás tareas para escribir en el mismo lote.

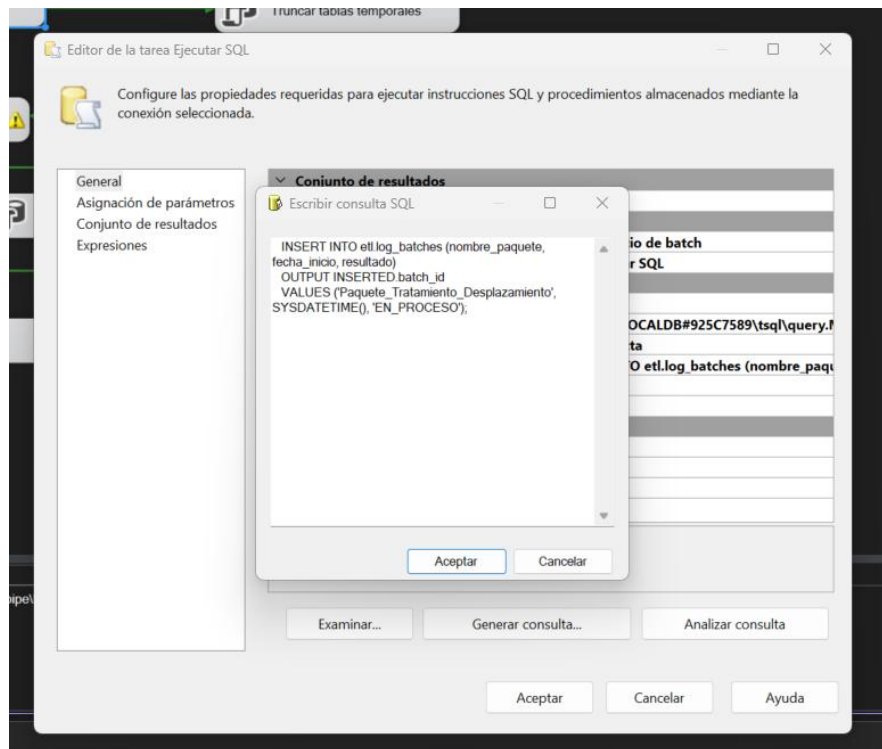


Figura 5. Editor de la tarea 'Registrar Inicio de batch' con la instrucción SQL real.

Truncar tablas temporales

Deja cada corrida en un estado limpio y repetible: vacía la zona de staging y las dos tablas temporales de lote antes de volver a extraer y transformar.

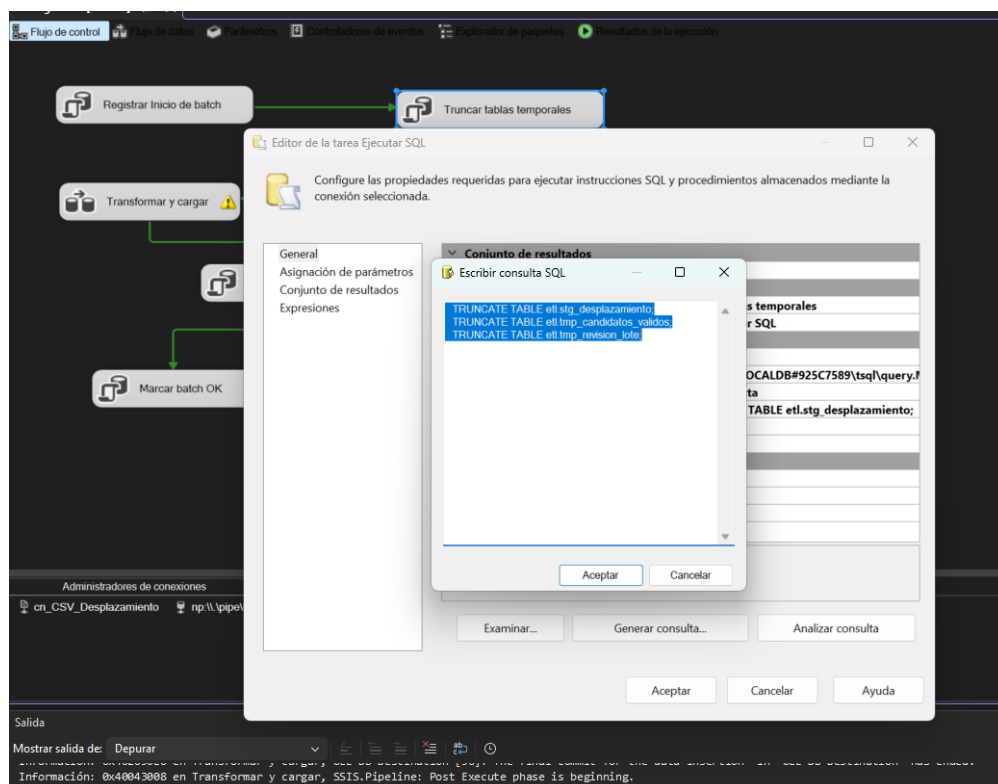


Figura 6. Editor de la tarea 'Truncar tablas temporales', con las tres instrucciones TRUNCATE TABLE reales.

Ejecutar MERGE idempotente

Llama al procedimiento almacenado que compara los candidatos válidos contra el destino y decide, por cada fila, si es un INSERT nuevo o un UPDATE de un registro ya existente. Es el mecanismo central que garantiza la idempotencia del proceso.

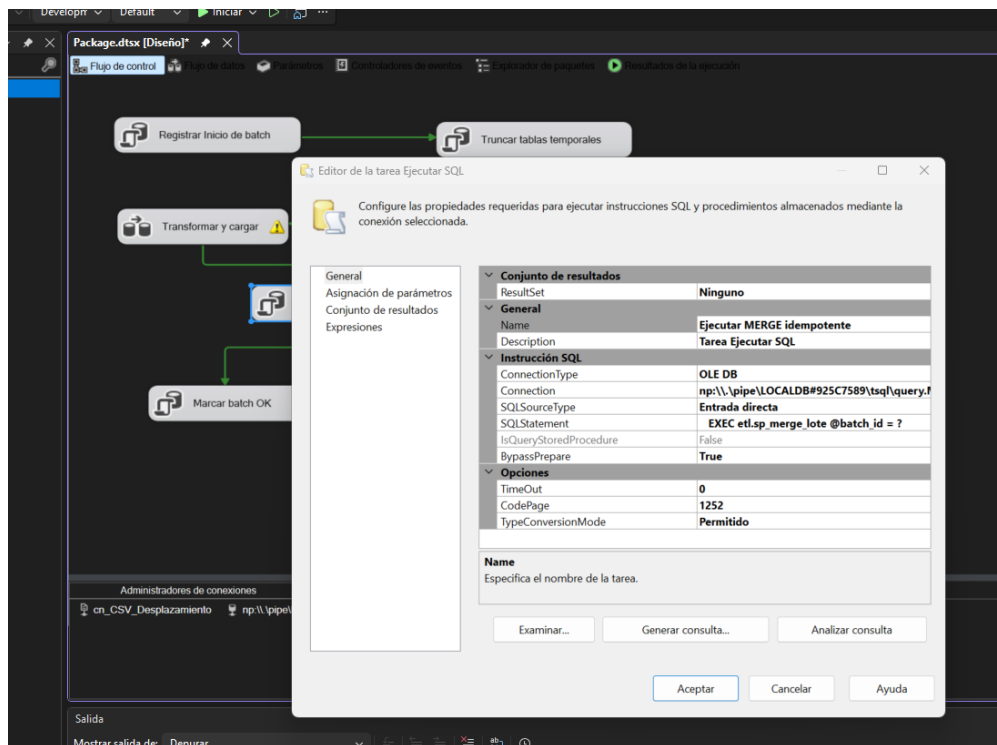


Figura 7. Editor de la tarea 'Ejecutar MERGE idempotente': `EXEC etl.sp_merge_lote @batch_id = ?`.

Marcar batch OK

Cierra el registro del lote en `etl.log_batches` con resultado = 'OK' y la fecha de fin, cuando el MERGE se ejecutó sin errores. Un Execute SQL Task equivalente, 'Marcar batch ERROR', hace lo mismo con resultado = 'ERROR' cuando la restricción de precedencia toma la rama de falla.

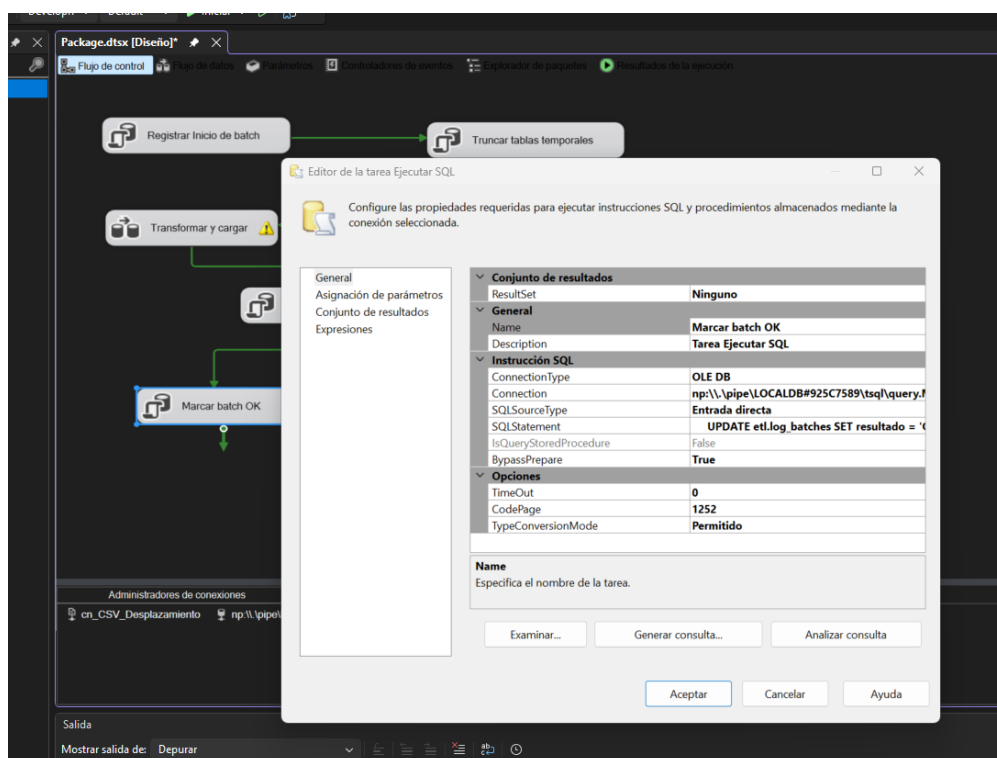


Figura 8. Editor de la tarea 'Marcar batch OK'.

4.1 Derived Column — 'Validaciones de texto'

Primer punto de limpieza: antes de convertir tipos, resuelve el caso de campos numéricos que llegan como cadena vacía (no NULL) desde el origen, y calcula las banderas de completitud y consistencia jerárquica.

- `es_sin_datos = ISNULL(personas_desplazadas) || TRIM(personas_desplazadas) == ""` — detecta el caso 'sin valor reportado'.
- `es_geo_inconsistente = (nivel == "Regional" && (ISNULL(departamento) || TRIM(departamento) == "")) || ((nivel == "Global" || nivel == "Nacional") && !ISNULL(departamento) && TRIM(departamento) != "")` — valida la jerarquía nivel/departamento/municipio.
- `personas_desplazadas_limpio = TRIM(personas_desplazadas) == "" ? NULL(DT_WSTR,30) : personas_desplazadas` — convierte cadena vacía en NULL real, requisito para que Data Conversion no falle.
- `desplazamiento_stock_limpio = TRIM(desplazamiento_stock) == "" ? NULL(DT_WSTR,30) : desplazamiento_stock` — mismo tratamiento para el stock.

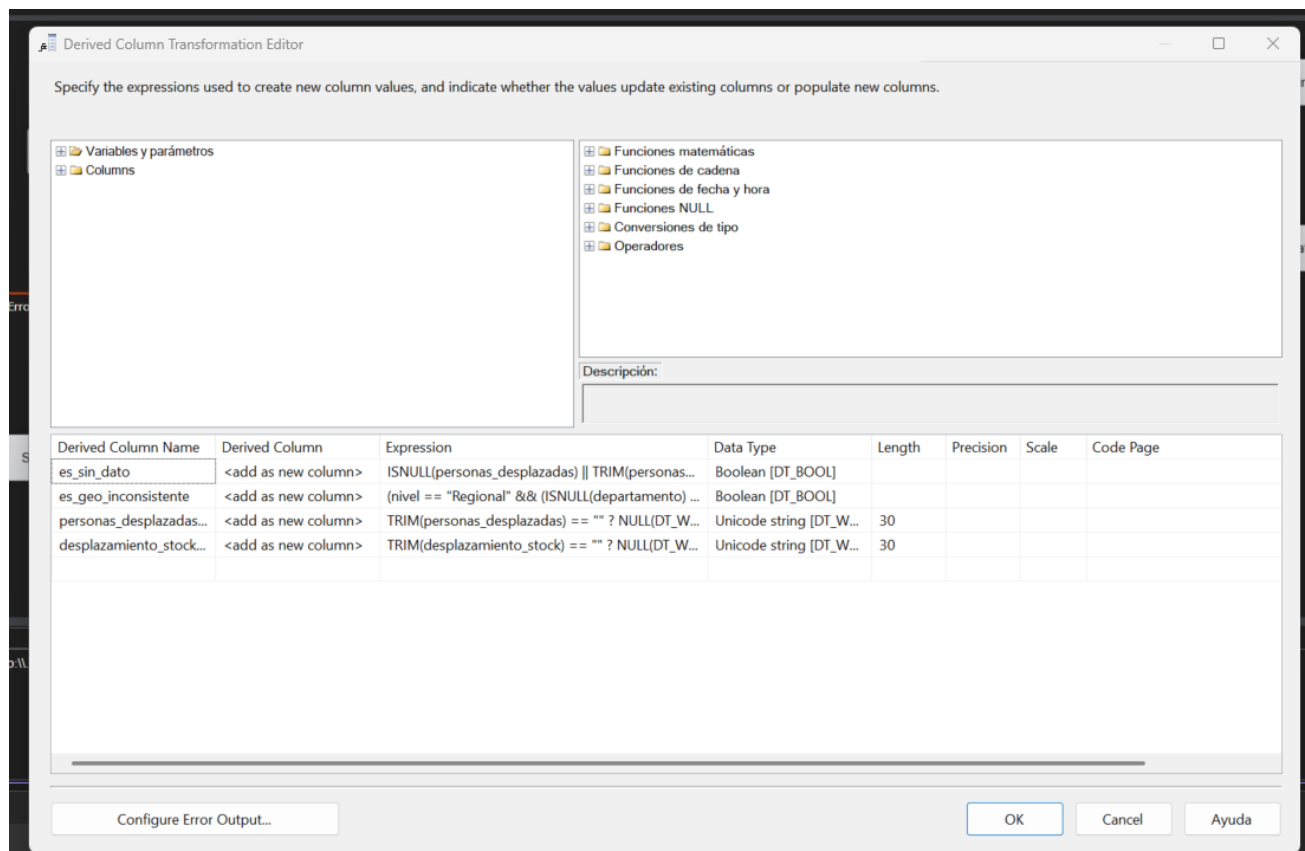


Figura 9. Derived Column — 'Validaciones de texto' configurado en SSIS/Visual Studio

4.2 Data Conversion — 'Convertir tipos'

Convierte explícitamente los tipos de dato leídos como texto desde el staging hacia los tipos definitivos; si la conversión falla, SSIS enruta la fila por el error output del propio componente ('Redirigir fila').

- `anio (DT_WSTR) → anio_conv (DT_I4)`.
- `personas_desplazadas_limpio → personas_desplazadas_conv (DT_NUMERIC, precisión 18, escala 2)`.
- `desplazamiento_stock_limpio → desplazamiento_stock_conv (DT_NUMERIC, precisión 18, escala 2)`, permite NULL.
- LocaleID del componente: 'Inglés (Estados Unidos)' — el dato origen usa punto como separador decimal; el LocaleID por defecto ('Español (Colombia)'), que espera coma) causaba fallos sistemáticos de conversión.

- Configuración de Error/Truncamiento: 'Redirigir fila' para las tres columnas — una fila que no puede convertirse no se pierde, se documenta como ERROR_CONVERSION_TIPO en la tabla de revisión.

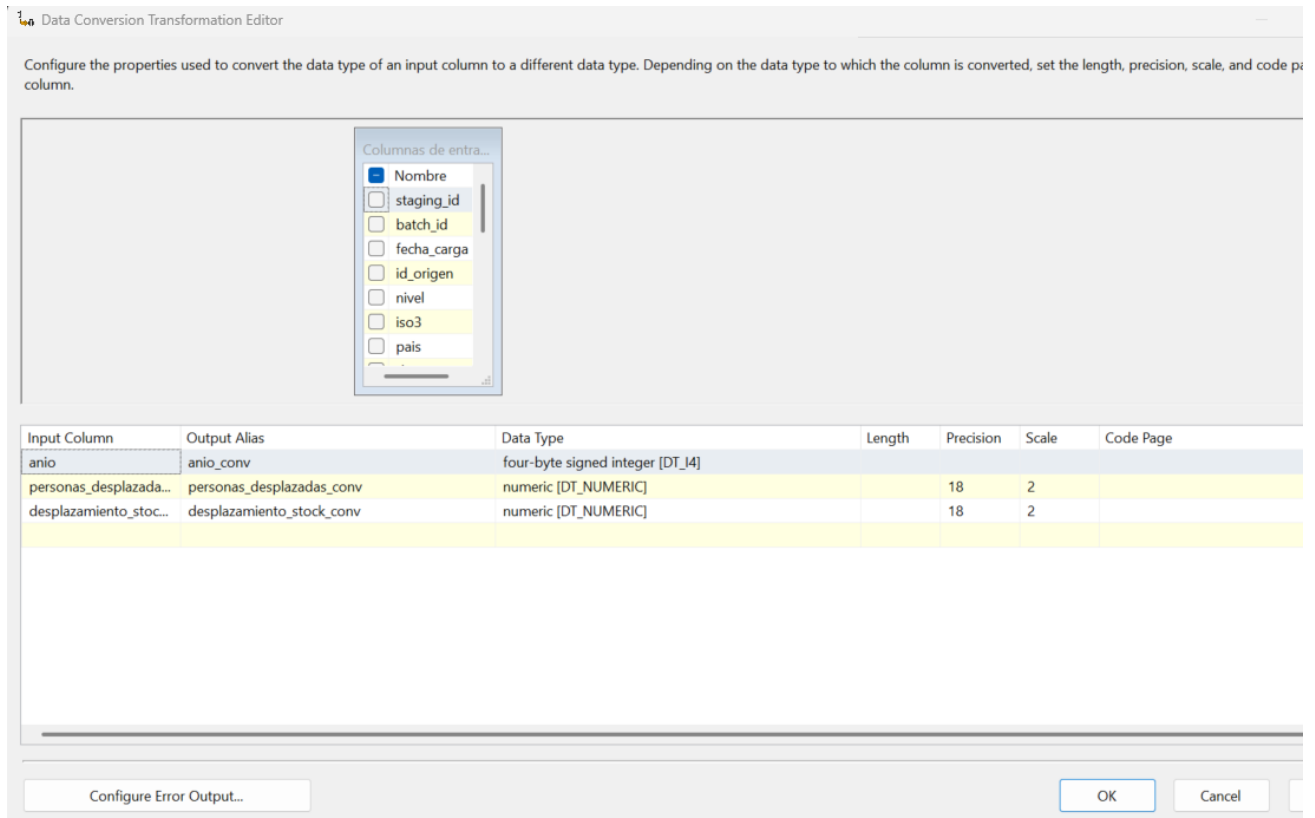


Figura 10. Data Conversion — 'Convertir tipos' configurado en SSDT/Visual Studio

4.3 Derived Column — cálculo de indicadores y llave de negocio

Con los tipos ya definitivos, calcula las últimas banderas de validación y construye la llave de negocio como texto plano.

- `es_negativo = personas_desplazadas_conv < 0` — valida que la cifra no sea negativa.
- `anio_fuera_de_rango = anio_conv < 1970 || anio_conv > 2026` — valida rango temporal.
- `clave_negocio_texto = (DT_WSTR,20)nivel + "|" + (DT_WSTR,10)iso3 + "|" + (ISNULL(departamento)?"":(DT_WSTR,150)departamento) + "|" + (ISNULL(municipio)?"":(DT_WSTR,150)municipio) + "|" + (DT_WSTR,10)anio_conv + "|" + (DT_WSTR,60)causa + "|" + (DT_WSTR,20)sexo + "|" + (DT_WSTR,20)grupo_edad + "|" + (DT_WSTR,250)fuelle` — identifica de forma única un hecho (nivel+país+división geográfica+año+causa+sexo+grupo de edad+fuelle). Se probó primero calculando un hash SHA-256 con HASHBYTES en un OLE DB Command, pero el proveedor OLE DB no logró preparar ese comando de forma confiable; se optó por la llave en texto plano, más simple y suficiente para este volumen.

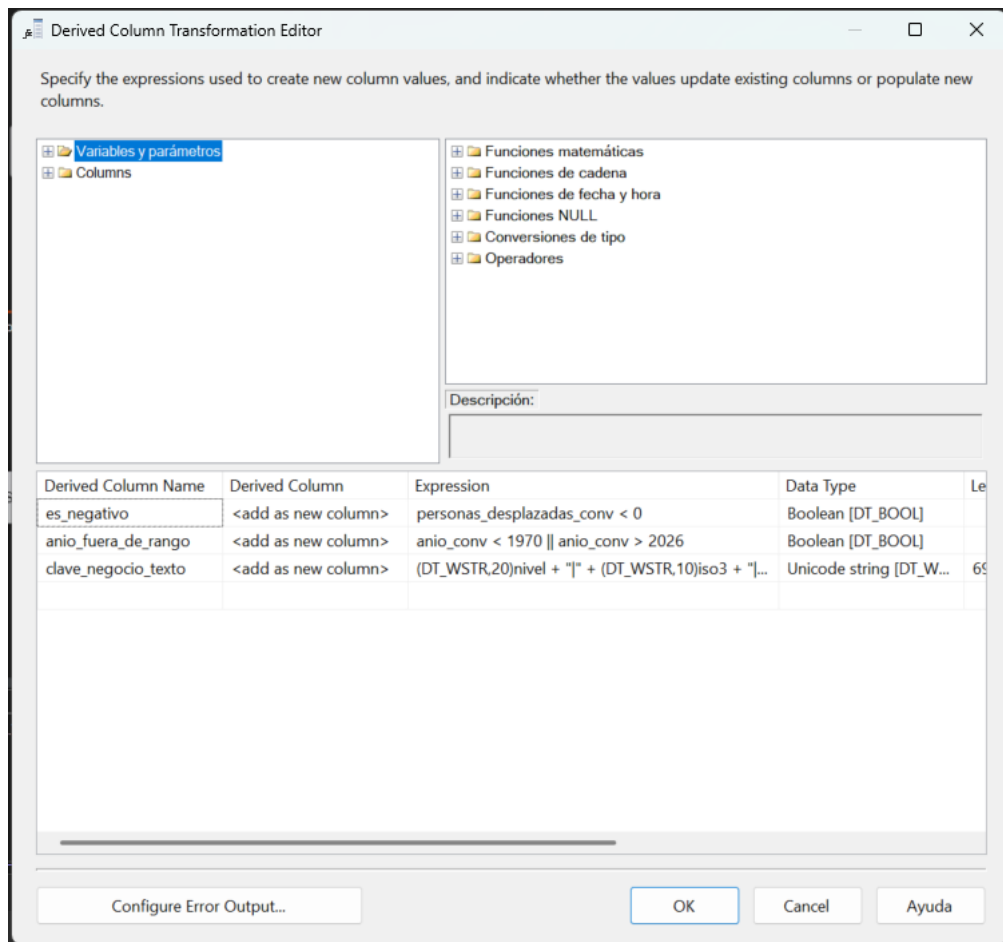


Figura 11. Derived Column — cálculo de indicadores y llave de negocio configurado

4.4 Lookup — dominio de sexo

Valida que el valor de sexo pertenezca al catálogo de referencia etl.ref_sexo, sin descartar las filas que no coinciden.

- Connection: OLE DB Connection Manager a MigracionDesplazamiento; tabla de referencia: SELECT valor FROM etl.ref_sexo (Full Cache).
- Columna de unión: sexo = valor.
- Specify how to handle rows with no matching entries → 'Redirigir filas a la salida de no coincidencia' (no 'Fail Component'): un valor fuera de dominio es un caso de negocio a revisar, no un error del paquete.
- Salida de coincidencia → Derived Column 'Sexo Válido' (sexo_invalido = (DT_BOOL)0). Salida sin coincidencia → Derived Column 'Sexo inválido' (sexo_invalido = (DT_BOOL)1).

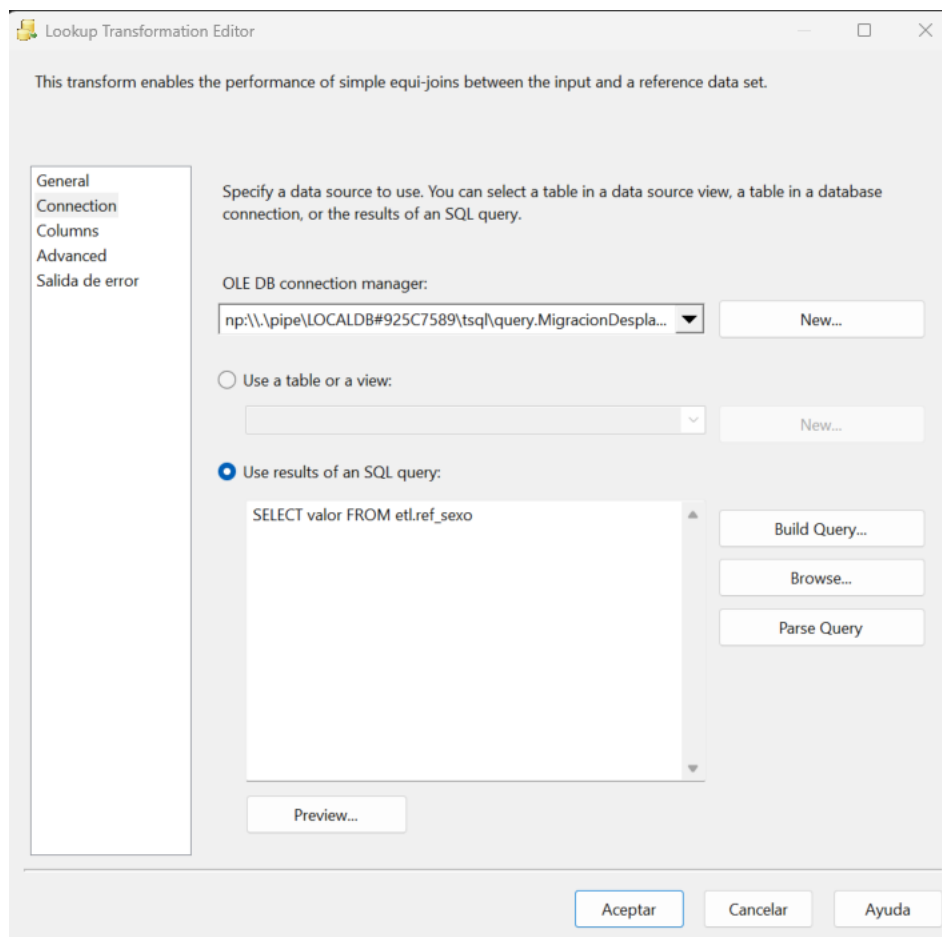


Figura 12. *Lookup — dominio de sexo configurado en SSIS/Visual Studio*

4.5 Union All

Reúne de nuevo en un único flujo las dos ramas que salieron del Lookup (coincidencia / no coincidencia), cada una ya con su bandera `sexo_invalido` calculada.

- Combina las columnas de ambas ramas en un solo buffer, condición necesaria para poder aplicar después una única lógica de Conditional Split sobre todas las filas.



Figura 13. Union All configurado en SSIS/Visual Studio

4.6 Derived Column — 'Calcular motivo de revisión'

Resuelve, con una sola expresión anidada, cuál es el motivo de revisión aplicable a la fila (si hay alguno), dando prioridad a las reglas en el orden que tiene más sentido de negocio.

- `motivo_revision = es_sin_dato ? "SIN_DATO_ORIGEN" : (es_negativo ? "VALOR_NEGATIVO" : (anio_fuera_de_rango ? "ANIO_FUERA_DE_RANGO" : (sexo_invalido ? "FUERA_DE_DOMINIO_SEXO" : (es_geo_inconsistente ? "INCONSISTENCIA_NIVEL_GEOGRAFIA" : NULL(DT_WSTR,60))))))`
- Si ninguna regla aplica, `motivo_revision` queda NULL y la fila sigue como candidata válida.

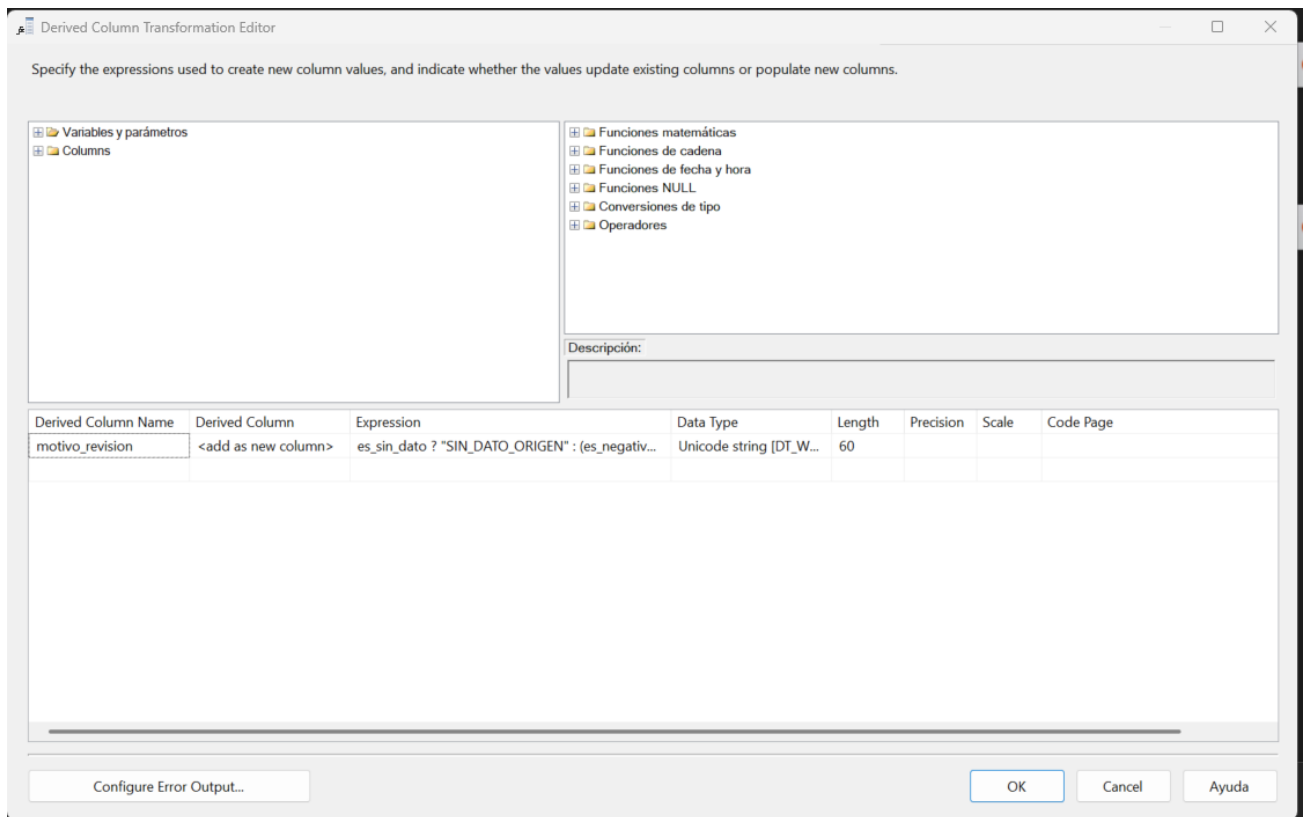


Figura 14. Derived Column — 'Calcular motivo de revisión'

4.7 Conditional Split

Separa cada fila en una de dos salidas según si motivo_revision quedó resuelto o no.

- Salida 'ARevision': !ISNULL(motivo_revision) — cualquier fila con un motivo de revisión calculado.
- Salida por defecto 'Validos': el resto de las filas, ya tipadas, sin banderas de error y con la llave de negocio calculada.
- La detección de duplicados NO ocurre aquí: toda fila 'Válida' se escribe en la tabla temporal y es el MERGE (paso 4 del Control Flow) el que decide si es nueva o ya existía en el destino.
- Los outliers (fuera de $3 \times \text{IQR}$) tampoco se separan en este Conditional Split: la columna outlier_iqr_extremo queda reservada en el esquema (BIT, default 0) para ese propósito; su cálculo automático se validó en la simulación Python y queda documentado como siguiente incremento (ver sección 5, hallazgo de la regla de atípicos).

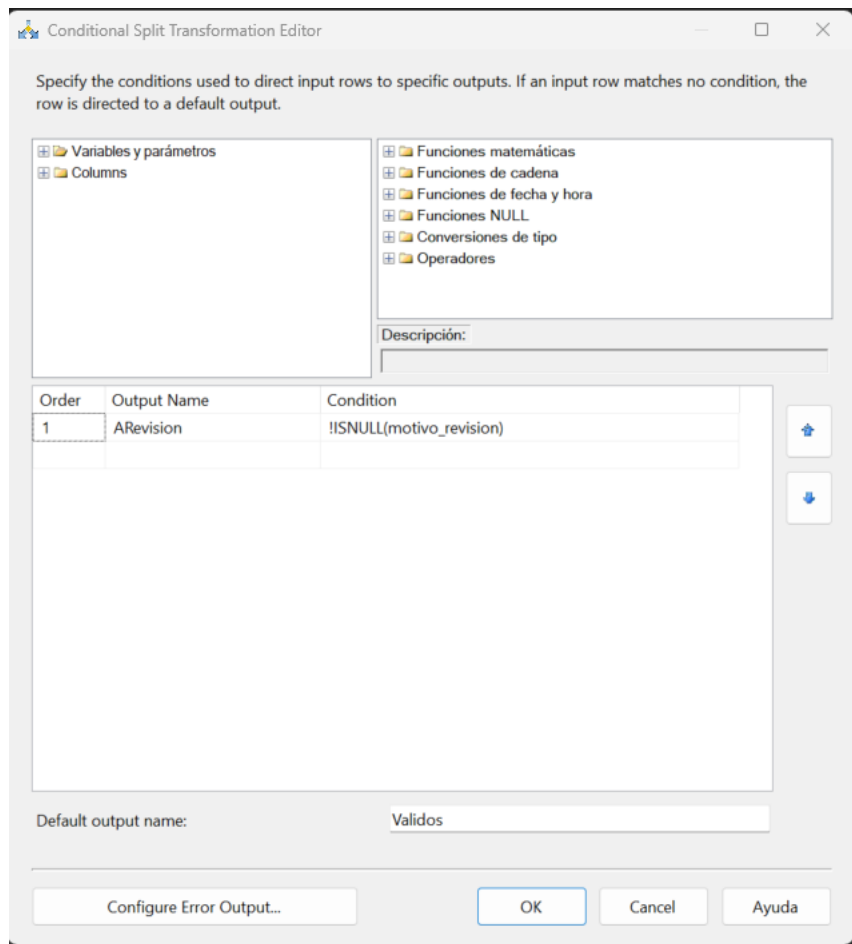
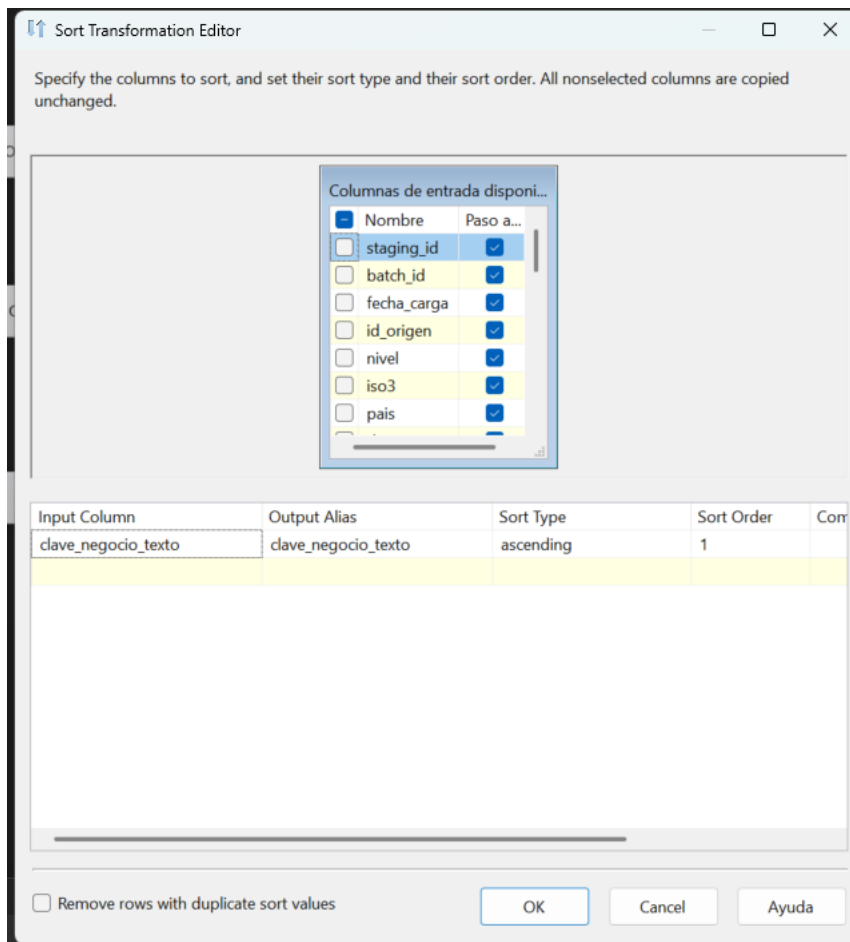


Figura 14. Conditional Split configurado

4.8 Sort

Ordena la salida 'Validos' por la llave de negocio antes de escribirla en el OLE DB Destination, dando un orden estable y determinístico a la carga.

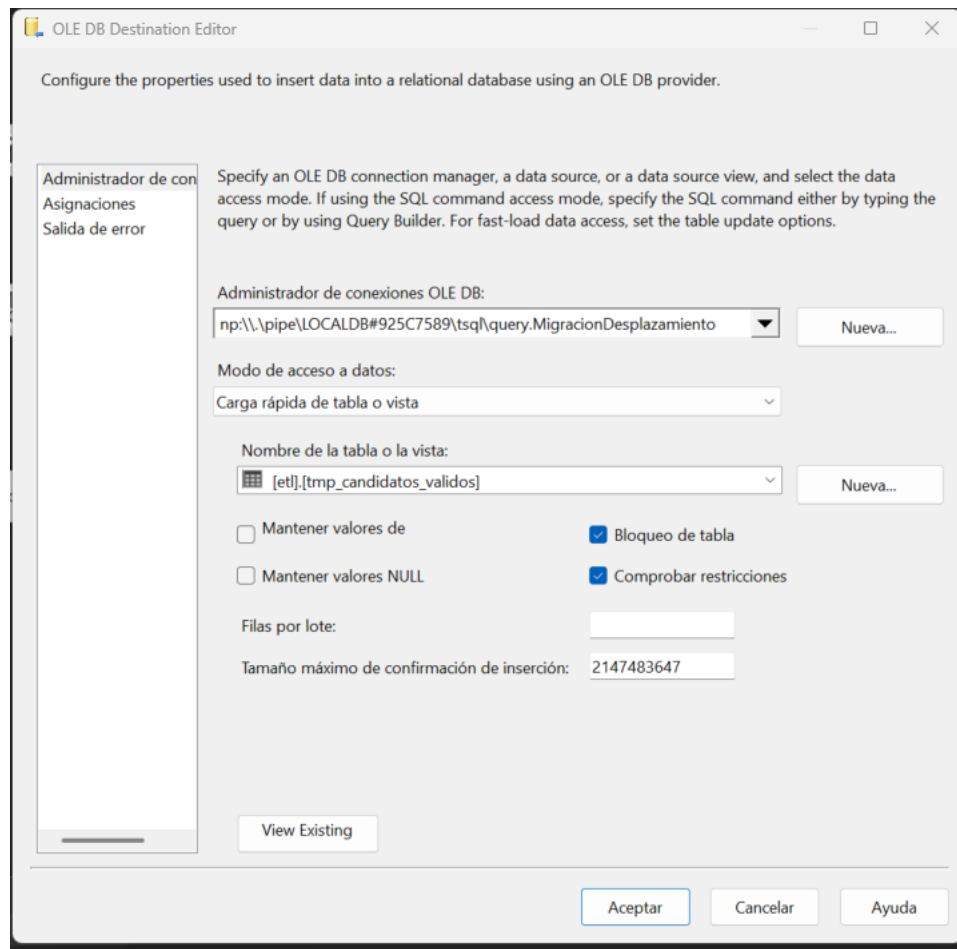
- Sort key: clave_negocio_texto, ascendente.
- 'Remove rows with duplicate sort values' se deja DESACTIVADO: el Sort solo ordena, no decide qué es o no duplicado — esa decisión la toma el MERGE contra el destino.



4.9 OLE DB Destination (dos salidas)

Escribe cada rama del Conditional Split en su tabla temporal correspondiente; ninguna de las dos escribe directamente sobre las tablas definitivas.

- Salida 'Validos' (ordenada) → etl.tmp_candidatos_validos. Mapeo manual: clave_negocio_texto → clave_negocio, anio_conv → anio, personas_desplazadas_conv → personas_desplazadas, desplazamiento_stock_conv → desplazamiento_stock; outlier_iqr_extremo se omite (usa el DEFAULT 0 de la tabla); el resto se mapea automáticamente por nombre.
- Salida 'ARevision' → etl.tmp_revision_lote. Mapeo manual: clave_negocio_texto → clave_negocio, motivo_revision → motivo_revision, y cada columna original (sin convertir) hacia su columna *_original correspondiente (nivel_original, iso3_original, pais_original, departamento_original, municipio_original, anio_original, causa_original, sexo_original, grupo_edad_original, personas_desplazadas_original, fuente_original), preservando el dato exactamente como llegó del origen.



4.10 MERGE idempotente (fuera del Data Flow)

El paso 5 del Control Flow ejecuta etl.sp_merge_lote (Execute SQL Task 'Ejecutar MERGE idempotente', ver Figura 7 en 4.0), que compara etl.tmp_candidatos_validos contra etl.desplazamiento_tratado por clave_negocio y decide la carga real.

- WHEN MATCHED (la clave_negocio ya existe en el destino) → UPDATE: se cuenta como duplicado, no se reinserta.
- WHEN NOT MATCHED (la clave_negocio es nueva) → INSERT: se cuenta como aceptado nuevo.
- La cláusula OUTPUT \$action INTO una tabla temporal permite separar, en el mismo Execute SQL Task, cuántas filas fueron INSERT y cuántas UPDATE, números que luego se escriben en etl.log_batches.
- Este es el mecanismo que garantiza la idempotencia: ejecutar el paquete varias veces con el mismo dataset nunca duplica el histórico ya cargado (verificado en la Iteración 3, ver sección 5).

4.11 Connection Manager

El Connection Manager usa autenticación integrada de Windows (Integrated Security) contra la instancia local de SQL Server; no se guarda usuario ni contraseña en el paquete ni en el repositorio. La cadena de conexión de ejemplo, sin credenciales, es:

```
Server=localhost;Database=MigracionDesplazamiento;Trusted_Connection=True;
```

5. Evidencia de las 3 iteraciones

Cada iteración se ejecutó sobre el mismo diseño de paquete, ajustando únicamente el alcance del origen y el catálogo de referencia de sexo entre la Iteración 1 y la Iteración 2, según lo que la propia ejecución reveló. Los conteos

siguientes están verificados: en cada iteración, aceptados + duplicados + enviados a revisión = recibidos, sin pérdida de registros.

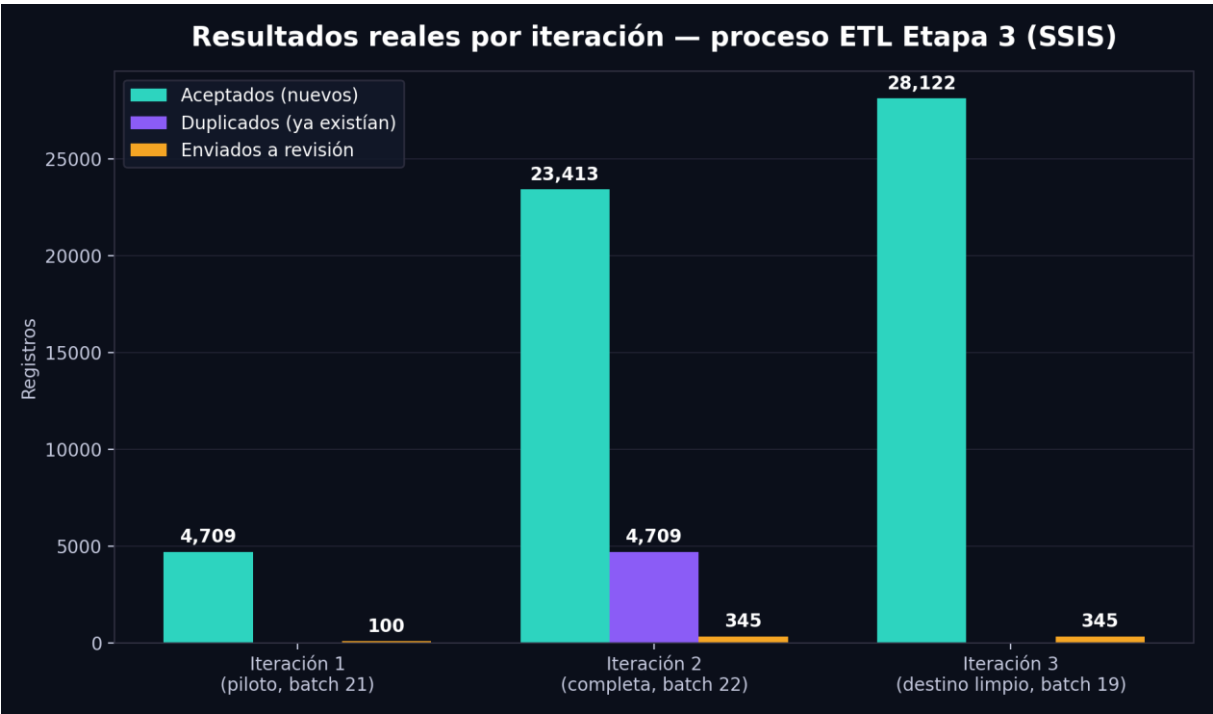


Figura 9. Comparación de resultados reales entre las tres iteraciones (calculada a partir de los batches 19-22 de etl.log_batches).

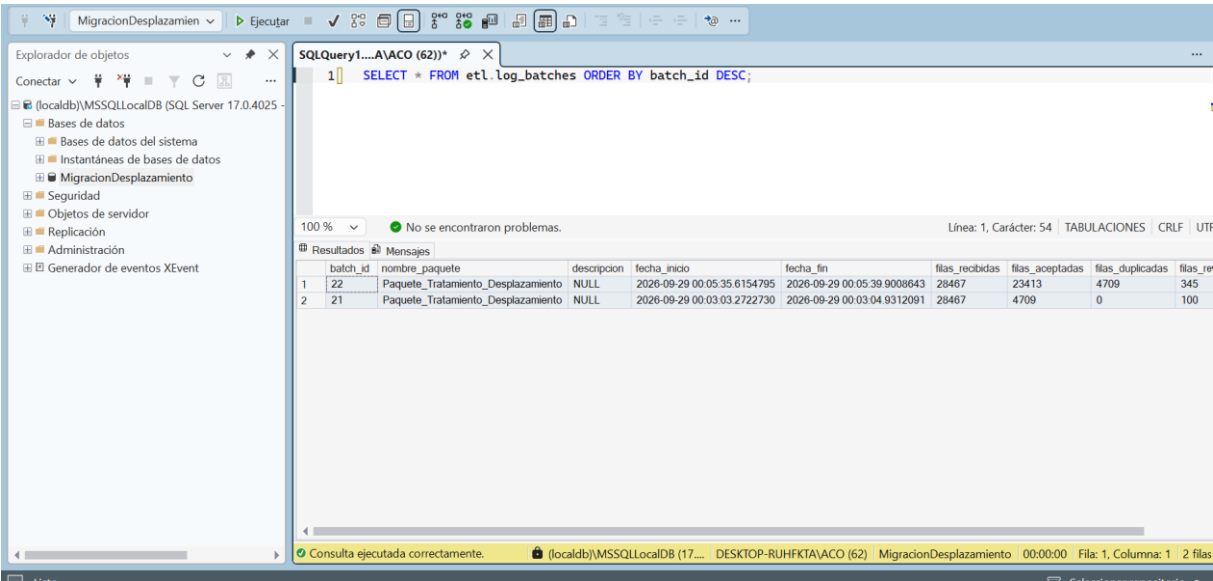


Figura 10. etl.log_batches en SSMS, mostrando los batches 21 (Iteración 1) y 22 (Iteración 2) tal como quedaron registrados (captura real).

5.1 Iteración 1 — Carga piloto (Colombia: Nacional + Regional) — batch_id 21

Primera corrida real del paquete SSIS, con el origen filtrado a Colombia (nivel IN ('Nacional','Regional')) y el catálogo ref_sexo deliberadamente incompleto (sin 'Both sexes'), para reproducir el hallazgo. El destino parte vacío.

Indicador	Valor
Recibidos	4809
Aceptados (nuevos)	4709
Duplicados (ya existían)	0
Enviados a revisión	100

Indicador	Valor
Marcados outlier (informativo)	0

Verificación de balance: $4709 + 0 + 100 = 4809 = 4809$ recibidos.

Hallazgos

- El catálogo de referencia de 'sexo' (ref_sexo) no incluía el valor 'Both sexes', y el componente Lookup mandó a revisión 100 registros con motivo FUERA_DE_DOMINIO_SEXO.
- El resto del lote (4.709 registros) pasó todas las demás validaciones (completitud, rango de año, negativos, consistencia jerárquica) sin problema.

Ajuste aplicado en la siguiente iteración: Completar ref_sexo con 'Both sexes' y quitar el filtro de Colombia para cargar el dataset mundial completo.

5.2 Iteración 2 — Carga completa (28.467 registros) — batch_id 22

Se corrige el catálogo ref_sexo y se quita el filtro de origen, cargando el dataset consolidado completo, sin limpiar el destino. Como incluye de nuevo los 4.809 registros de Colombia de la Iteración 1, el MERGE los detecta como ya existentes y los actualiza en vez de reinsertarlos.

Indicador	Valor
Recibidos	28467
Aceptados (nuevos)	23413
Duplicados (ya existían)	4709
Enviados a revisión	345
Marcados outlier (informativo)	0

Verificación de balance: $23413 + 4709 + 345 = 28467 = 28467$ recibidos.

Hallazgos

- Los 4.709 registros de Colombia aceptados en la Iteración 1 se identificaron correctamente como duplicados (misma clave_negocio) vía el MERGE (WHEN MATCHED → UPDATE) y no se reinsertaron.
- Con el catálogo de sexo corregido, 0 registros se rechazaron por FUERA_DE_DOMINIO_SEXO en esta corrida (antes eran 100 en el lote piloto).
- Los 345 registros enviados a revisión corresponden exclusivamente a valores nulos de personas_desplazadas (motivo SIN_DATO_ORIGEN), tal como se documentó en la Etapa 2. Balance verificado: $23.413 + 4.709 + 345 = 28.467$.

Ajuste aplicado en la siguiente iteración: Las reglas ya quedaron estables; en la Iteración 3 se confirma el resultado final desde un destino limpio y se prueba la idempotencia del proceso.

5.3 Iteración 3 — Recarga completa desde destino limpio + prueba de idempotencia — batch_id 19 y 20

Se trunca el destino y se vuelve a correr el paquete completo con las reglas ya maduras (v3 = v2) para tener una foto definitiva (batch 19). Después, sin limpiar nada, se ejecuta exactamente el mismo lote una segunda vez (batch 20) para comprobar que no se generan duplicados.

Indicador	Valor
Recibidos	28467
Aceptados (nuevos)	28122
Duplicados (ya existían)	0
Enviados a revisión	345

Indicador	Valor
Marcados outlier (informativo)	0

Verificación de balance: $28122 + 0 + 345 = 28467 = 28467$ recibidos.

Hallazgos

- Con el destino limpio, el resultado final es: 28122 aceptados, 345 en revisión, 0 duplicados (0, porque el destino partió vacío).
- Al re-ejecutar el mismo lote de 28467 registros sin limpiar el destino: 0 registros nuevos insertados y 28122 detectados como ya existentes.
- El tamaño del destino antes y después de la segunda ejecución no cambió ($28122 = 28122$), lo que confirma que el proceso es idempotente.

Prueba de idempotencia (mismo lote ejecutado una segunda vez, sin limpiar el destino)

Indicador	Valor
Recibidos	28467
Ya existían (duplicados)	28122
Nuevos insertados	0
Enviados a revisión	345
Tamaño del destino antes	28122
Tamaño del destino después	28122

Resultado: el tamaño del destino no cambió ($28122 = 28122$) y 0 filas nuevas se insertaron en la segunda corrida. El proceso es idempotente.

6. Manejo de errores y tratamiento de registros en revisión

Un registro nunca se descarta silenciosamente. Cuando una fila no cumple alguna regla, el Conditional Split la enruta hacia etl.tmp_revision_lote, que conserva las columnas *_original (nivel_original, iso3_original, anio_original, personas_desplazadas_original, etc.) exactamente como llegaron del staging, junto con motivo_revision indicando cuál regla falló. Esa tabla se integra por MERGE hacia etl.desplazamiento_revision, que además agrega resuelto (bit) y comentario_resolucion, para que un analista de datos pueda dar seguimiento manual sin perder el registro original.

Motivos de revisión utilizados

- SIN_DATO_ORIGEN — personas_desplazadas llegó nulo desde la fuente.
- VALOR_NEGATIVO — personas_desplazadas o desplazamiento_stock es negativo.
- ERROR_CONVERSION_TIPO — la Data Conversion no pudo convertir anio o los montos.
- FUERA_DE_DOMINIO_SEXO — el valor de sexo no está en el catálogo de referencia (evidenciado y corregido entre la Iteración 1 y la 2).
- INCONSISTENCIA_NIVEL_GEOGRAFIA — un registro Regional sin municipio, o un registro Global/Nacional con municipio.
- ANIO_FUERA_DE_RANGO — anio fuera de 1970–año actual.

7. Comparación de calidad antes y después

Indicador	Antes	Después
Compleitud (variable central)	99.8%	100%
Validez	100%	100%

Indicador	Antes	Después
Duplicados en destino	—	0

Los 345 registros sin dato no se 'completaron' artificialmente; se aislaron en una tabla de revisión separada, por lo que la tabla destino queda 100% completa en su variable central, con trazabilidad total de lo excluido.

Verificación total: 28.467 registros recibidos = 28.122 en destino + 345 en revisión. Sin el control de llave de negocio, cada recarga completa del dataset habría duplicado el histórico completo: 28.467 (si no existiera el control de llave de negocio, cada recarga completa duplicaría todo el histórico); con el control activo (verificado en la Iteración 3, batch 20), los duplicados detectados y NO reinsertados en cada recarga posterior fueron 0.

8. Enlaces de entrega

- Aplicación Flask publicada: <https://dromeroh14.pythonanywhere.com/etapa3>
- Repositorio en GitHub (rama feature/etapa-3): <https://github.com/DROMEROH14/migracion-desplazamiento-humano>
- Recursos de esta etapa en el repositorio: etapa3/sql/ (scripts de base de datos), etapa3/etl_simulation.py (simulación verificada de la lógica del Data Flow), etapa3/*.dtsx (paquete SSIS).
- Video de demostración (máx. 4 minutos, grabación individual): enlace publicado junto con este informe en la sección Etapa 3 de la aplicación.